

一分钟视频生成与测试时训练

Karan Dalal^{*4} Daniel Koceja^{*2} Gashon Hussein^{*2} Jiarui Xu^{*1,3} Yue Zhao^{†5} Youjin Song^{†2}
 Shihao Han¹ Ka Chun Cheung¹ Jan Kautz¹ Carlos Guestrin² Tatsunori Hashimoto² Sanmi Koyejo²
 Yejin Choi¹ Yu Sun^{1,2} Xiaolong Wang^{1,3}
¹NVIDIA ²Stanford University ³UCSD ⁴UC Berkeley ⁵UT Austin

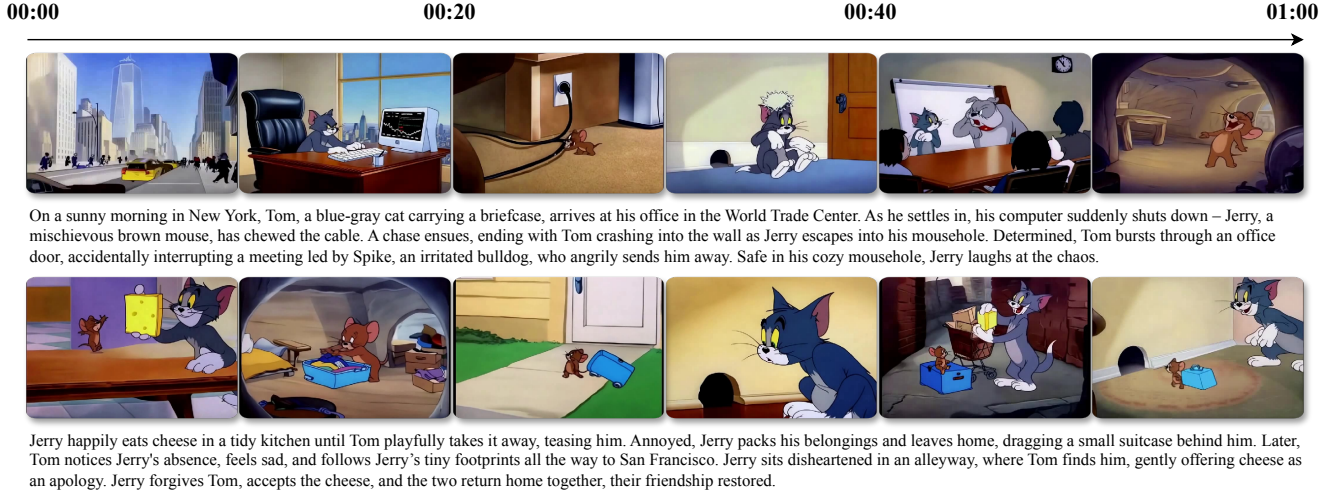


图 1. TTT 层使预训练的扩散 Transformer 能够从文本故事板生成一分钟视频。我们以《猫和老鼠》动画片作为概念验证。这些视频讲述复杂故事，场景连贯，动作动态丰富。每个视频均由模型直接一次性生成，无需剪辑、拼接或后期处理。每个故事均为全新创作。

Abstract

当前，Transformer 仍难以生成一分钟视频，因为自注意力层在处理长上下文时效率低下。Mamba 层等替代方案难以处理复杂的多场景故事，因为其隐藏状态表达能力较弱。我们尝试使用测试时训练（TTT）层，其隐藏状态本身可以是神经网络，因此表达能力更强。将 TTT 层加入预训练的 Transformer 后，模型能够从文本故事板生成一分钟视频。作为概念验证，我们基于《猫和老鼠》动画片构建了一个数据集。与 Mamba 2、门控 DeltaNet 和滑动窗口注意力层等基线相比，TTT 层生成的视频连贯性显著提升，能够讲述复杂故事，在每方法 100 个视频的人工评估中领先 34 个 Elo 分。

^{*}Joint first authors. [†] Joint second authors.

Accepted to The IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) 2025

尽管结果令人鼓舞，但视频中仍存在伪影，这可能是由于预训练的 5B 模型能力有限所致。我们实现的效率也有待提高。受资源限制，我们仅实验了一分钟视频，但该方法可扩展至更长视频和更复杂的故事。示例视频、代码和标注可在以下地址获取：

<https://test-time-training.github.io/video-dit>

1. Introduction

尽管在视觉和物理真实感方面取得了显著进展，最先进的视频 Transformer 仍主要生成缺乏复杂故事情节的单场景短视频片段。截至撰写本文时（2025 年 3 月），公开视频生成 API 的最大时长分别为：Sora（OpenAI）20 秒，MovieGen（Meta）16 秒，Ray 2 10 秒。

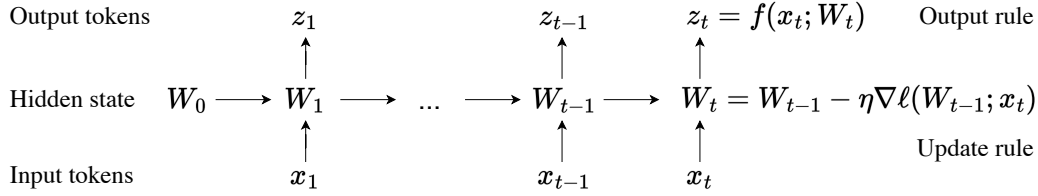


图 2. 所有 RNN 层均可表示为根据更新规则进行状态转移的隐藏状态。[43] 中的核心思想是使隐藏状态本身成为一个模型 f ，其权重为 W ，更新规则为对自监督损失 ℓ 的梯度步。因此，在测试序列上更新隐藏状态等价于在测试时训练模型 f 。这一过程称为测试时训练 (TTT)，被编程到 TTT 层中。图和图注取自 [43]。

(Luma)，以及 Veo 2 (谷歌) 的 8 个。这些应用程序编程接口均无法自主生成复杂的多场景故事。这些技术局限背后的一个根本性挑战是长上下文，因为变换器中自注意力层的成本随上下文长度呈二次方增长。对于具有动态运动的视频生成而言，这一挑战尤为严峻，因为其上下文难以通过分词器轻松压缩。使用标准分词器，我们每个一分钟视频在上下文中需要超过 30 万个标记。若采用自注意力，生成一分钟视频所需时间将比生成 20 个各 3 秒的视频长 $11\times$ ，训练时间也将长 $12\times$ 。为应对这一挑战，近期视频生成研究探索了循环神经网络层作为自注意力的高效替代方案，因为其成本随上下文长度线性增长 [47]。现代循环神经网络层，尤其是线性注意力变体 [23, 37]，如曼巴 [8, 12] 和德尔塔网络 [35, 53]，在自然语言任务中已展现出令人瞩目的成果。然而，我们尚未见到由循环神经网络生成的具有复杂故事或动态运动的长视频。[47] 中的视频（链接）分辨率高且时长一分钟，但仅包含单一场景和缓慢运动，更不用说复杂故事了。我们认为，这些循环神经网络层生成的视频复杂度较低，是因为其隐藏状态表达能力不足。循环神经网络层只能将过去的标记存储到固定大小的隐藏状态中，对于曼巴和德尔塔网络等线性注意力变体而言，该隐藏状态仅为一个矩阵。将数十万个向量压缩到一个秩仅为数千的矩阵中，本质上极具挑战性。因此，这些循环神经网络层难以记住远距离标记之间的深层关系。我们尝试了另一类循环神经网络层，其隐藏状态本身可以是神经网络。具体而言，我们使用两层多层感知机，其隐藏单元数量比线性注意力变体中的线性（矩阵）隐藏状态多 $2\times$ 倍，且具有更丰富的非线性。由于神经网络隐藏状态即使在测试序列上也会通过训练进行更新，这些新层被称为测试时训练层 [43]。

我们从预训练的扩散 Transformer (CogVideo-X 5B [19]) 出发，该模型原本只能以 16 帧每秒生成 3 秒的短视频片段（或以 8 帧每秒生成 6 秒）。随后，我们加入从零初始化的 TTT 层，并对该模型进行微调，使其能够根据文本故事板生成一分钟的视频。我们将自注意力层限制在 3 秒的片段内，以控制其计算成本。仅经过初步的系统优化，我们的训练运行在 256 块 H100 上相当于耗时 50 小时。我们基于 ≈ 7 小时的《猫和老鼠》动画片构建了一个文本到视频数据集，并配有经过人工标注的故事板。我们有意将研究范围限定在这一特定领域，以便快速进行科研迭代。作为概念验证，我们的数据集强调复杂、多场景、长程且具有动态运动的故事，这些方面仍需取得进展；而在视觉和物理真实感方面，由于已取得显著进展，我们较少强调。我们相信，在这一特定领域提升长上下文能力将迁移到通用视频生成中。与 Mamba 2 [8]、Gated DeltaNet [53] 和滑动窗口注意力层等强基线相比，TTT 层生成的视频更加连贯，能够讲述具有动态运动的复杂故事，在对每种方法 100 个视频进行的人工评估中领先 34 个 Elo 分。作为参考，GPT-4o 在 LMSys Chatbot Arena [6] 中比 GPT-4 Turbo 高出 29 个 Elo 分。示例视频、代码和标注可在以下地址获取：<https://test-time-training.github.io/video-dit>

2. 测试时训练层

按照标准做法 [44, 54]，每个视频被预处理为 T 个标记的序列，其中 T 由其时长和分辨率决定。本节回顾用于通用序列建模的测试时训练 (TTT) 层，部分采用了 [43] 第 2 节的阐述。我们首先讨论如何以因果方式（按时间顺序）处理一般输入序列。第 3 节讨论如何在非因果主干中通过沿相反方向调用 RNN 层来使用它们。

2.1. TTT 作为隐藏状态的更新

所有循环神经网络（RNN）层都将历史上下文压缩到一个固定大小的隐藏状态中。这种压缩带来两个后果。一方面，将输入标记 x_t 映射到输出标记 z_t 是高效的，因为更新规则和输出规则对每个标记都只需常数时间。另一方面，循环神经网络层记忆长上下文的能力受限于其隐藏状态所能存储的信息量。[43] 的目标是设计具有表达力隐藏状态的循环神经网络层，以压缩海量上下文。作为启发，他们观察到自监督学习可以将大规模训练集压缩到机器学习模型的权重中。[43] 的核心思想是利用自监督学习将历史上下文 $x_1, \dots, x_t$ 压缩到隐藏状态 W_t 中，方法是上下文视为无标签数据集，将隐藏状态视为机器学习模型的权重 f 。图 2 所示的更新规则是在某个自监督损失 ℓ 上进行一步梯度下降：

$$W_t = W_{t-1} - \eta \nabla \ell(W_{t-1}; x_t), \quad (1)$$

学习率为 η 。直观地说，输出标记就是 f 使用更新后的权重 W_t 对 x_t 所做的预测：

$$z_t = f(x_t; W_t). \quad (2)$$

ℓ 的一种选择是重建 x_t 本身。为了使学习问题具有非平凡性，可以先将 x_t 处理成损坏的输入 $\tilde{x}_t$ （见 2.2 小节），然后优化：

$$\ell(W; x_t) = \|f(\tilde{x}_t; W) - x_t\|^2. \quad (3)$$

与去噪自编码器类似 [46]， f 需要发现 x_t 各维度之间的相关性，才能从部分信息 $\tilde{x}_t$ 中重建它。与其他循环神经网络层和自注意力一样，这种将输入序列 $x_1, \dots, x_T$ 映射到输出序列 $z_1, \dots, z_T$ 的算法可以被编程到序列建模层的前向传播中。即使在测试时，该层仍会为每个输入序列训练不同的权重序列 $W_1, \dots, W_T$ 。因此，它被称为测试时训练（TTT）层。从概念上讲，对 $\nabla \ell$ 调用反向传播意味着对梯度求梯度——这是元学习中已被充分研究的技术。测试时训练层与循环神经网络层和自注意力具有相同的接口，因此可以在任何更大的网络架构中被替换。[43] 将训练更大的网络称为外循环，将每个测试时训练层内对 W 的训练称为内循环。

2.2. 为测试时训练学习自监督任务

可以说，测试时训练最重要的部分是由 ℓ 指定的自监督任务。[43] 没有根据人类先验手工设计自监督任务，而是采用了一种更

端到端方法，将其作为外循环的一部分进行学习。从方程 3 中的朴素重建任务出发，他们使用低秩投影 $\tilde{x}_t = \theta_K x_t$ ，其中 θ_K 是在外循环中可学习的矩阵。此外，也许 x_t 中的信息并非全部值得记忆，因此重建标签也可以是低秩投影 $\theta_V x_t$ 而非 x_t 。总之，[43] 中的自监督损失为：

$$\ell(W; x_t) = \|f(\theta_K x_t; W) - \theta_V x_t\|^2. \quad (4)$$

最后，由于 $\theta_K x_t$ 的维度少于 x_t ，[43] 不能再使用方程 2 中的输出规则。因此他们进行另一个投影 $\theta_Q x_t$ ，并将输出规则改为：

$$z_t = f(\theta_Q x_t; W_t). \quad (5)$$

注意，在内循环中，仅优化 W ，因此将其写为 ℓ 的参数； θ 是该内循环损失函数的“超参数”。 $\theta_K, \theta_V, \theta_Q$ 在外循环中优化，类似于自注意力的查询、键和值参数。

2.3. TTT-MLP 实例化

遵循 [43]，我们将内循环模型 f 实例化为 f_{MLP} 的包装器：一个类似于变换器中使用的两层多层感知机。具体地，隐藏维度为输入维度的 $4 \times$ 倍，后接 GELU 激活 [16]。为了在 TTT 期间获得更好的稳定性， f 始终包含层归一化和残差连接。即，

$$f(x) = x + \text{LN}(f_{\text{MLP}}(x)).$$

具有此 f 的 TTT 层称为 TTT-MLP，是本文通篇的默认实例化。在第 4 节中，我们还将 TTT-Linear（上述 f 包装线性模型）实例化为基线。

3. 方法

在高层次上，我们的方法简单地将 TTT 层添加到预训练的扩散 Transformer 中，并在带有文本标注的长视频上对其进行微调。在实际层面上，使该方法有效涉及许多设计选择。

3.1. 架构

预训练扩散 Transformer。 本文提出的添加 TTT 层后进行微调的方法，原则上可适用于任何骨干架构。我们选择扩散 Transformer（Diffusion Transformer）[32] 作为初步演示，因为它是视频生成中最流行的架构。由于在视频上预训练扩散 Transformer 的成本过高，我们从名为 CogVideo-X 5B [19] 的预训练检查点开始。

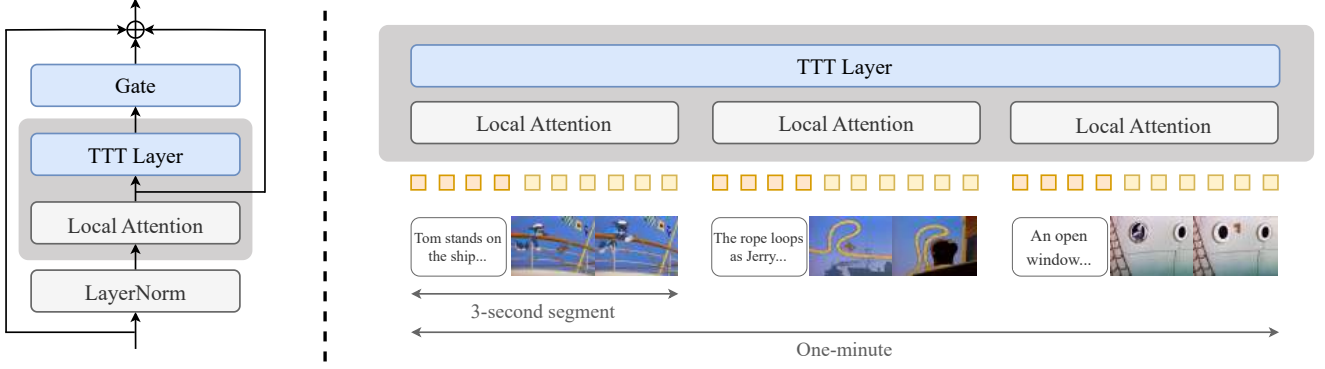


图 3. 本文方法概览。左：修改后的架构在每个注意力层之后添加一个带可学习门控的 TTT 层。参见第 3.1 小节。右：整体流程创建由 3 秒片段组成的输入序列。该结构使我们能够在片段上局部应用自注意力层，并在整个序列上全局应用 TTT 层。参见第 3.2 小节。

门控。 给定输入序列 $X = (x_1, \dots, x_T)$ ，其中每个标记 $x_t \in \mathbb{R}^d$ ，TTT 层产生输出序列 $Z = (z_1, \dots, z_T) = \text{TTT}(X)$ 。每个 $z_t \in \mathbb{R}^d$ 遵循第 2 节中方程 1、4 和 5 描述的递推关系。直接将 TTT 层插入预训练网络会在微调开始时（TTT 层随机初始化时）显著恶化其预测。为避免这种退化，我们按照标准做法 [1]，使用可学习向量 $\alpha \in \mathbb{R}^d$ 对 TTT 进行门控：

$$\text{gate}(\text{TTT}, X; \alpha) = \tanh(\alpha) \otimes \text{TTT}(X) + X, \quad (6)$$

其中 $\tanh(\alpha) \in (-1, 1)^d$ 与 $Z = \text{TTT}(X)$ 中的每个 z_t 逐元素相乘。我们将 α 中的所有值初始化为 0.1，因此在微调开始时 $\tanh(\alpha)$ 中的值接近 0 (≈ 0.1)。这种 α 的初始化允许 TTT 仍然对 $\text{gate}(\text{TTT}, X; \alpha)$ 有贡献，而不会显著覆盖 X 。双向。扩散模型（包括 CogVideo-X）是非因果的，这意味着输出标记 z_t 可以依赖于所有 $x_1, \dots, x_T$ ，而不仅仅是过去的标记 $x_1, \dots, x_t$ 。为了以非因果方式使用 TTT 层，我们应用一种称为双向 [30] 的标准技巧。给定一个将 $X = (x_1, \dots, x_T)$ 在时间上反转的算子 $\text{rev}(X) = (x_T, \dots, x_1)$ ，我们定义

$$\text{TTT}'(X) = \text{rev}(\text{TTT}(\text{rev}(X))). \quad (7)$$

由于 rev 被应用两次， $\text{TTT}'(X)$ 仍按时间顺序排列。但其内部的 TTT 层现在以逆时间顺序扫描 X 。修改后的架构。标准 Transformer（包括 CogVideo-X）包含交错的序列建模模块和 MLP 块。具体来说，标准序列建模模块接受输入序列 X 并产生

$$X' = \text{self_attn}(\text{LN}(X)) \quad (8)$$

$$Y = X' + X, \quad (9)$$

其中 LN 为层归一化（Layer Norm）¹， $X' + X$ 构成残差连接。本文仅修改序列建模模块，架构其余部分保持不变。每个修改后的模块如图 3 左图所示，从方程 8 中的 X' 继续并产生

$$Z = \text{gate}(\text{TTT}, X'; \alpha), \quad (10)$$

$$Z' = \text{gate}(\text{TTT}', Z; \beta), \quad (11)$$

$$Y = Z' + X. \quad (12)$$

注意 TTT' 仅再次调用 TTT，因此它们共享相同的底层参数 $\theta_K, \theta_V, \theta_Q$ 。但对于门控，方程 10 和 11 使用不同的参数 α 和 β 。

3.2. 整体流程

本小节讨论如何为本文架构创建输入标记序列，以及每个序列如何分段处理。除即将讨论的前两种文本格式外，其余内容均适用于微调和推理。本文流程如图 3 右图所示。

场景与片段。 本文将视频结构化包含多个场景，² 每个场景包含一个或多个 3 秒片段。本文使用 3 秒片段作为文本-视频配对的原子单元，原因有三：• 原始预训练 CogVideo-X 的最大生成长度为 3 秒。

- 《猫和老鼠》剧集中大多数场景的长度至少为 3 秒。
- 给定 3 秒片段，构建多阶段数据集（第 3.3 小节）最为方便。

文本提示的格式。 在推理时，用户可以采用以下三种格式中的任意一种来编写长视频的文本提示，

¹ 扩散变换器（如 CogVideo-X）使用自适应层归一化 [32]。² 场景的宽松定义是“电影中动作发生在同一地点或属于某一特定类型的部分。”（牛津词典）

以下列出的格式按细节递增的顺序排列。参见附录中的图 8 以查看每种格式的示例。

- **格式 1**：用 5-8 句话对情节进行简短总结。部分示例如图 1 所示。

- **格式 2**：一个更详细的情节，大约 20 句话，每句话大致对应一个 3 秒的片段。句子可以标注为属于某些场景或场景组，但这些标注仅作为建议。

- **格式 3**：分镜脚本。每个 3 秒片段由一段 3-5 句话的段落描述，包含背景颜色和摄像机运动等细节。一个或多个段落组成的组被严格强制属于某些场景，使用关键词<scene start>和<scene end>。

在微调 and 推理过程中，我们的文本分词器的实际输入始终采用格式 3。格式之间的转换由 Claude 3.7 Sonnet 按 $1 \rightarrow 2 \rightarrow 3$ ³ 的顺序执行。对于微调，我们的人工标注已经是格式 3，如第 3.3 小节所述。

从文本到序列。 在原始 CogVideo-X 为每个视频对输入文本进行分词后，它将文本标记与带噪声的视频标记拼接起来，形成变换器的输入序列。为了生成视频，我们对每个 3 秒片段独立应用相同的过程。具体来说，给定一个格式 3 的分镜脚本，包含 n 个段落，我们首先生成 n 个序列片段，每个片段包含从相应段落提取的文本标记，后跟视频标记。然后将所有 n 个序列片段拼接在一起形成输入序列，该序列现在具有交错的文本和视频标记。

局部注意力，全局 TTT。 CogVideo-X 使用自注意力层对最大长度为 3 秒的每个视频全局处理整个输入序列，但全局注意力对于长视频变得低效。为了避免增加自注意力层的上下文长度，我们使它们对每个 3 秒片段局部化，独立地关注每个 n 序列片段。⁴ TTT 层全局处理整个输入序列，因为它们长上下文效率高。

3.3. 微调方案与数据集

多阶段上下文扩展。 遵循大语言模型的标准实践 [51]，我们将修改后架构的上下文长度分五个阶段扩展到一个分钟。首先，我们在《猫和老鼠》的 3 秒片段上微调整个预训练模型，使其适应这一领域。新参数（特别是 TTT 层和门控中的参数）被

³ 我们观察到，直接从格式 1 转换到格式 3 会导致在微调数据集中遵循格式 3 人工标注风格的能力变差。⁴ 作为预处理步骤的产物，序列片段实际上有 1 个潜在帧（1350 个标记）的重叠。

在此阶段分配了更高的学习率。在接下来的四个阶段中，我们分别在 9 秒、18 秒、30 秒和最终 63 秒的视频上进行微调。为了避免遗忘太多预训练中的世界知识，我们仅微调 TTT 层、门控和自注意力层，在这四个阶段使用较低的学习率。详细配方见附录 A。

原始视频上的超分辨率。 我们从 1940 年至 1948 年间发布的 81 集《猫和老鼠》开始。每集约 5 分钟，所有剧集总计约 7 小时。原始视频分辨率不一，以现代标准来看普遍较差。我们在原始视频上运行视频超分辨率模型 [49]，为数据集生成视觉增强的视频，共享分辨率为 720×480 。

多阶段数据集。 遵循第 3.2 小节讨论的结构，我们首先让人类标注员将每集分解为场景，然后从每个场景中提取 3 秒片段。接下来，我们让人类标注员为每个 3 秒片段撰写详细段落。⁵ 阶段 1 直接在这些片段上微调。为了为最后四个阶段创建数据，我们将连续的 3 秒片段拼接成 9 秒、18 秒、30 秒和 63 秒的视频，连同它们的文本标注。场景边界由第 3.2 小节中的相同关键词标记。因此，所有训练视频的标注均为格式 3。

3.4. 非因果序列的并行化

第 2 节讨论的更新规则无法在序列中的标记之间直接并行化，因为计算 W_t 需要 $\nabla \ell(W_{t-1}; x_t)$ ，而这又需要 W_{t-1} 。为了实现并行化，我们一次更新 W 在 b 个标记上，这 [43] 称为内循环小批量。在本文中，我们设置 $b = 64$ 。具体而言，对于小批量 $i = 1, \dots, T/b$ （假设 T 是整数倍 b ），

$$W_{ib} = W_{(i-1)b} - \frac{\eta}{b} \sum_{t=(i-1)b+1}^{ib} \nabla \ell(W_{(i-1)b}; x_t). \quad (13)$$

由于序列是非因果的，我们随后使用 W_{ib} 为小批量 i 中的所有时间步生成输出标记，其中

$$z_t = f(W_{ib}; x_t), \quad t = (i-1)b+1, \dots, ib. \quad (14)$$

注意， $W_{(i-1)b+1}, \dots, W_{ib-1}$ 不再需要。经过这一修改后， f 可以并行处理一个（内循环）小批量的标记，类似于常规多层感知机处理一个（外循环）小批量训练数据的方式。作为额外的好处，我们观察到跨标记平均梯度可以降低方差，并稳定对 W 的每次更新。

⁵ 每个段落包含 1-2 句描述背景的句子、1-2 句描述角色的句子，以及 2 句描述动作和摄像机运动的句子。平均每个段落包含 98 个单词，相当于 132 个标记。

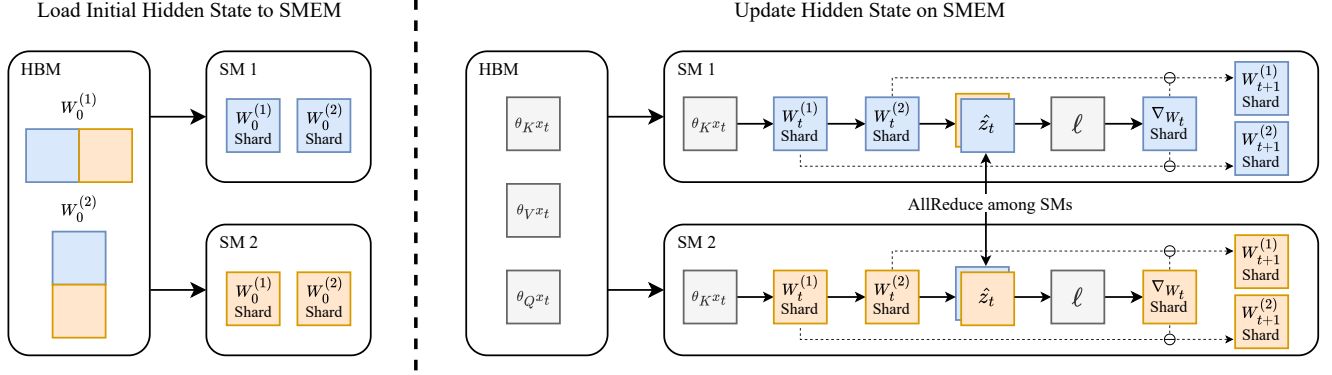


图 4. 片上张量并行, 详见第 3.5 小节。左图: 为了减少每个流式多处理器上 TTT-MLP 所需的内存, 我们将隐藏状态 $W^{(1)}$ 和 $W^{(2)}$ 分片到各个流式多处理器上, 仅在初始加载和最终输出时在 HBM 与 SMEM 之间传输。右图: 我们完全在片上更新隐藏状态, 并利用 NVIDIA Hopper GPU 架构上的 DSMEM 特性在流式多处理器之间对中间激活进行全归约。

3.5. 片上张量并行

在 GPU 上高效实现 TTT-MLP 需要特殊设计以利用其内存层次结构。GPU 上的芯片称为流式多处理器 (Streaming Multiprocessor, SM), 类似于 CPU 上的核心。GPU 上的所有 SM 共享一个相对较慢但容量大的全局内存, 称为高带宽内存 (HBM), 而每个 SM 拥有一个快速但容量小的片上内存, 称为共享内存 (SMEM)。GPU 上 SMEM 与 HBM 之间的频繁数据传输会显著损害整体效率。Mamba 和自注意力层 (Flash 注意力 [9]) 的高效实现使用核函数融合来最小化这种传输。这些实现的高层思想是将输入和初始状态加载到每个 SMEM 中, 完全在片上执行计算, 并仅将最终输出写回 HBM。然而, TTT-MLP 的隐藏状态, 即两层 MLP f 的权重 $W^{(1)}$ 和 $W^{(2)}$, 太大而无法存储在单个 SM 的 SMEM 中 (当与输入和激活结合时)。为了减少每个 SM 所需的内存, 我们使用张量并行 [39] 在 SM 之间分片 $W^{(1)}$ 和 $W^{(2)}$, 如图 4 所示。类似于大型 MLP 层可以在多个 GPU 的 HBM 之间分片和训练, 我们现在将相同的想法应用于多个 SM 的 SMEM 之间, 将每个 SM 视为 GPU 的类比。我们使用 NVIDIA Hopper GPU 架构上的 DSMEM 特性来实现 SM 之间的全归约。我们的核函数更多细节在附录 B 中讨论。我们的实现显著提高了效率, 因为隐藏状态和激活现在仅在初始加载和最终输出期间从 HBM 读取和写入。作为一般原则, 如果模型架构 f 可以使用标准张量并行在 GPU 之间分片, 那么当 f 用作隐藏状态时, 相同的分片策略可以应用于 SM 之间。

4. Evaluation

我们对 TTT-MLP 和五个基线进行多轴基准的人工评估, 所有基线均具有线性复杂度: 局部注意力、TTT-Linear、Mamba 2、门控 DeltaNet 和滑动窗口注意力层。

4.1. Baselines

除局部注意力外, 所有基线均使用第 3.1 小节中的方法添加到相同的预训练 CogVideo-X 5B 中; 它们修改后的架构均具有 7.2B 参数。所有基线使用第 3.3 小节和附录 A 中相同的微调方案。接下来我们详细讨论基线。

- **局部注意力:** 对原始架构不做修改, 在每个 3 秒片段上独立执行自注意力。

- **测试时训练 (TTT) -Linear**
 $x + \text{LN}(f_{\text{Linear}}(x))$ [43]: 一个实例化 $f(x) =$ 的 TTT 层, 其中 f_{Linear} 是一个线性模型。

- Mamba 2 [8]: 一种具有矩阵隐藏状态的现代 RNN 层, 其隐藏状态比 TTT-Linear 中的大 $\approx 4\times$ 倍, 但比 TTT-MLP 中的小 $\approx 2\times$ 倍。

- **门控 DeltaNet** [53]: DeltaNet [52] 和 Mamba 2 的扩展, 具有改进的更新规则。

- **滑动窗口注意力** [3]: 具有 8192 个标记 (约 1.5 秒视频) 固定窗口的自注意力。

4.2. 评估维度与协议

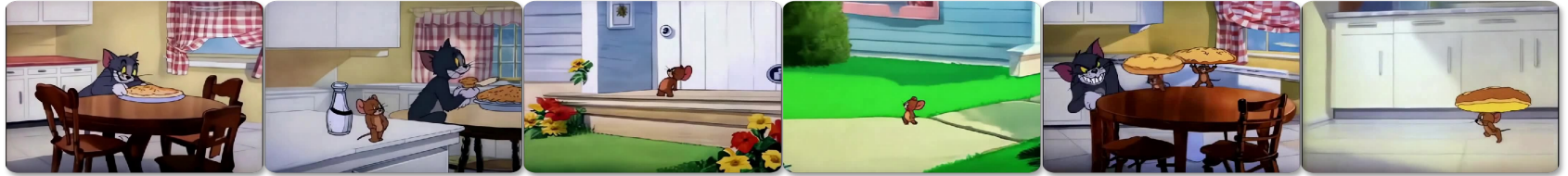
从 MovieGen [44] 的六个评估维度中, 我们采用与我们的领域相关的四个维度进行人工评估。⁶

⁶ 在 MovieGen 的六个维度中, 我们省略了不适用于卡通的“真实感”。我们还省略了“运动完整性”, 该维度“衡量输出视频是否包含足够的运动”, 因为我们领域中的所有视频都具有高度动态的运动。我们将“帧一致性”调整为“时序一致性”, 以同时涵盖跨场景的一致性。

Tom is happily eating an apple pie at the kitchen table. Jerry looks longingly wishing he had some. Jerry goes outside the front door of the house and rings the doorbell. While Tom comes to open the door, Jerry runs around the back to the kitchen. Jerry steals Tom's apple pie. Jerry runs to his mouse hole carrying the pie, while Tom is chasing him. Just as Tom is about to catch Jerry, he makes it through the mouse hole and Tom slams into the wall.



TTT-MLP preserves temporal consistency over scene changes and across angles, producing smooth, high-quality actions.



Sliding-window attention alters the kitchen environment, changes the house color, and duplicates Jerry stealing the pie.



Gated DeltaNet lacks temporal consistency across different angles of Tom but maintains the kitchen environment in later frames.



Mamba 2 distorts Tom's appearance as he growls and chases Jerry but maintains a similar kitchen environment throughout the video.

Figure 5. Video frames comparing TTT-MLP against Gated DeltaNet and sliding-window attention, the leading baselines in our human evaluation. TTT-MLP demonstrates better scene consistency by preserving details across transitions and better motion naturalness by accurately depicting complex actions.

	Text following	Motion naturalness	Aesthetics	Temporal consistency	Average
Mamba 2	985	976	963	988	978
Gated DeltaNet	983	984	993	1004	991
Sliding window	1016	1000	1006	975	999
TTT-MLP	1014	1039	1037	1042	1033

表 1. 一分钟视频的人工评估结果。TTT-MLP 平均比第二好的方法高出 34 个 Elo 分。提升最大的维度是场景一致性 (+38) 和运动平滑度 (+39)。作为参考，在 Chatbot Arena 中，GPT-4 比 GPT-3.5 Turbo 高 46 个 Elo 分，GPT-4o 比 GPT-4 Turbo 高 29 个 Elo 分 [6]。

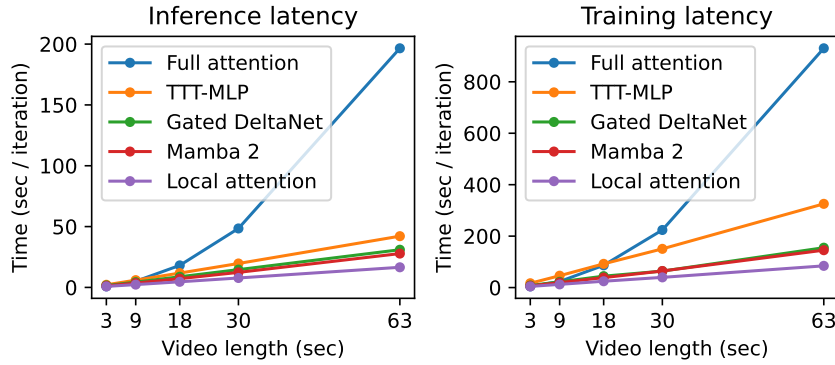


图 6. 对于 63 秒的视频，如第 1 节所述，使用全注意力（超过 30 万 token）进行推理所需时间比局部注意力长 11 $\times$ ，训练所需时间比局部注意力长 12 $\times$ 。TTT-MLP 分别需要 2.5 $\times$ 和 3.8 $\times$ ——比全注意力高效得多，但仍不如例如 Gated DeltaNet 高效，后者在推理和训练中所需时间均比局部注意力长 1.8 $\times$ 。

- **Text following:** “alignment with the provided prompt.”
- **运动自然度:** “自然的肢体运动、面部表情以及对物理规律的遵循。不自然或诡异怪诞的运动将被扣分。”
- **美学:** “有趣且引人入胜的内容、光线、色彩和镜头效果。”
- **时序一致性:** 场景内部以及场景之间均保持一致。

引用的描述来自 MovieGen [44]。我们的评估基于盲测比较中的成对偏好，因为直接对长视频进行评分或一次性对多个视频进行排序具有挑战性。具体而言，评估者会从上述四个维度中随机获得一个维度，以及一对共享相同情节的视频，然后被要求指出在该维度上表现更好的视频。为了收集视频池，我们首先使用 Claude 3.7 Sonnet（采用第 3.2 小节讨论的 Format 1 $\rightarrow$ 2 $\rightarrow$ 3）采样 100 个情节，然后为每个情节的每种方法生成一个视频。生成视频的方法始终对评估者保密。我们的评估者通过 prolific.com 招募，筛选条件为：居住在美国、母语为英语、年龄在 18 至 35 岁之间、至少有 100 次先前提交记录且批准率至少为 98%。网站上披露的评估者人口统计信息如下。

- **性别:** 50.78% 男性，47.66% 女性，1.56% 其他。
- **种族:** 57.03% 白人，23.44% 黑人，10.94% 混血，5.47% 亚裔，以及 3.12% 其他。基于这些信息，我们认为我们的评估者

constitute a representative sample of the U.S. population.

4.3. 结果

我们使用 LMSys Chatbot Arena [6] 中的 Elo 系统聚合成对偏好。Elo 分数如表 1 所示。TTT-MLP 平均比第二好的方法高出 34 个 Elo 分。作为参考，在 LMSys Chatbot Arena [6] 中，GPT-4 比 GPT-3.5 Turbo 高出 46 个 Elo 分（1163 对 1117），GPT-4o 比 GPT-4 Turbo 高出 29 个 Elo 分（1285 对 1256），因此我们 34 分的提升在实际中是有意义的。⁷ 图 5 比较了 TTT-MLP 和基线方法生成的示例视频帧。图 5 中展示的视频可以在项目网站上访问：
<https://test-time-training.github.io/video-dit>

18 秒淘汰赛。请注意，局部注意力和 TTT-Linear 未出现在表 1 中。为避免在所有方法上评估更长视频带来的更高成本，我们首先按照第 4.2 小节讨论的相同流程，使用 18 秒视频进行了一轮淘汰赛。这一轮淘汰了表现最差的局部注意力，以及表现不如 TTT-MLP 的 TTT-Linear。淘汰赛的结果见附录中的表 3。

⁷ <https://lmarena.ai/>，访问于 2025 年 3 月 20 日。所考虑的模型包括 GPT-4o-2024-05-13、GPT-4-Turbo-2024-04-09、GPT-4-0613 和 GPT-3.5-Turbo-0613。

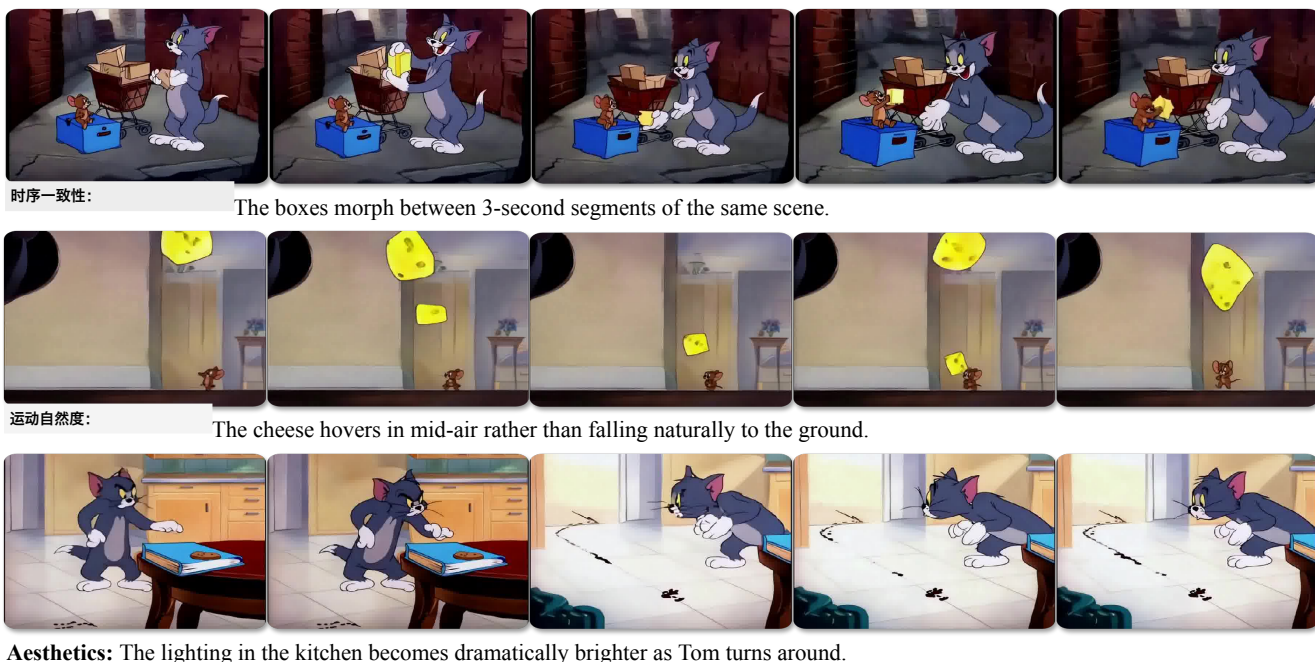


图 7. TTT-MLP 生成的视频中的伪影。时序一致性：物体有时在 3 秒片段的边界处发生形变，可能是因为扩散模型在不同片段中采样自不同的模式。运动自然度：物体有时会不自然地漂浮，因为重力效应未被正确建模。美学：光照变化并不总是与动作一致，除非明确提示。复杂的摄像机运动（如视差）有时被不准确地描绘。

4.4. 局限性

短上下文。对于上述 18 秒淘汰赛，Gated DeltaNet 平均表现最佳，领先 Mamba 2 27 个 Elo 分，领先 TTT-MLP 28 个 Elo 分（见附录中的表 3）。对于 18 秒视频，上下文长度约为 10 万 token。该评估展示了具有线性（矩阵）隐藏状态的 RNN 层（如 Gated DeltaNet 和 Mamba 2）仍然是最有效的场景。此外，18 秒和 63 秒视频的评估结果均表明，Gated DeltaNet 相比 Mamba 2 有显著改进。

实际运行时间。即使在应用了第 3.4 和 3.5 小节中的改进之后，TTT-MLP 的效率仍然低于 Gated DeltaNet 和 Mamba 2。这一局限性在图 6 中得到了突出显示，例如，使用 TTT-MLP 进行推理和训练分别比使用 Gated DeltaNet 慢 1.4× 和 2.1×。第 6 节讨论了 TTT-MLP 核函数的两个潜在改进方向，以提高效率。需要注意的是，在我们的应用中，训练效率并不是一个重要的关注点，因为 RNN 层是在预训练之后集成的，而预训练占据了整体训练预算的大部分。RNN 层的训练效率仅在微调期间才相关，而微调本身只占预算的一小部分。相比之下，推理效率则更有意义。

视频伪影。生成的 63 秒视频作为概念验证展示了明显的潜力，但仍然包含显著的伪影，尤其是在运动自然性和美学方面。图 7 展示了与我们三个评估维度相对应的伪影示例。我们观察到，具有这类伪影的视频并非 TTT-MLP 所特有，而是所有方法中普遍存在的。这些伪影可能是预训练的 CogVideo-X 5B 模型能力有限所导致的。例如，由原始 CogVideo-X 生成的视频（链接）在运动自然性和美学方面似乎也有限。

5. 相关工作

现代 RNN 层，尤其是线性注意力变体 [23, 37]，如 Mamba[8, 12] 和 DeltaNet[35, 52]，在自然语言任务中表现出了令人印象深刻的性能。受其成功以及快速权重编程器 [7, 21, 24, 36] 思想的启发，[43] 提出了可扩展且实用的方法，使隐藏状态更大且非线性，从而更具表达力。近期工作 [2] 开发了更大且更非线性的隐藏状态，并使用更复杂的优化技术进行更新。[43] 中的相关工作部分详细讨论了 TTT 层的灵感来源。[48] 对 RNN 层的最新发展进行了很好的概述。

长视频建模。一些早期工作 [40] 通过训练生成对抗网络 (GAN) [11, 22], 基于当前帧和运动向量预测下一帧, 从而生成视频。由于自回归 (AR) 和基于扩散的方法 [13, 25, 44, 54] 的近期进展, 生成质量已显著提升。TATS[10] 在变换器 (Transformer) 上提出滑动窗口注意力, 以生成超过训练长度的视频。Phenaki[45] 以类似的自回归方式工作, 但每一帧由 MaskGIT[4] 生成。预训练的扩散模型可以通过级联 [15, 50, 55]、流式 [17] 和添加过渡 [5] 来扩展生成更长的视频。故事综合方法如 [20, 26, 28, 29, 31, 33] 生成与文本故事中各个句子对应的图像或视频序列。例如, Craft[14] 通过检索生成复杂场景的视频, StoryDiffusion[56] 利用扩散来改善帧间过渡的平滑性。虽然与文本到视频生成相关, 故事综合方法通常需要在其流程中加入额外组件以保持跨场景的一致性, 而这些组件并非端到端处理。

6. 未来工作

本文概述了几个有前景的未来研究方向。

更快的实现。我们当前的 TTT-MLP 核函数受限于寄存器溢出和异步指令的次优排序。通过最小化寄存器压力并开发更具编译器感知的异步操作实现, 效率可能进一步提升。

更好的集成。使用双向和学习门控只是将 TTT 层集成到预训练模型中的一种可能策略。更好的策略应进一步提升生成质量并加速微调。其他视频生成骨干网络, 如自回归模型, 可能需要不同的集成策略。

更长视频与更大隐藏状态。本文方法有望扩展至生成更长的视频, 且保持线性复杂度。我们认为, 实现该目标的关键在于将隐藏状态实例化为比两层多层感知机 (MLP) 规模更大的神经网络。例如, f 本身可以是一个变换器 (Transformer)。

致谢。感谢超曲实验室 (Hyperbolic Labs) 提供的计算支持, 感谢邓云天在实验运行方面的帮助, 以及阿里安·辛加尔 (Aaryan Singhal)、阿尔琼·维克拉姆 (Arjun Vikram) 和本·斯佩克特 (Ben Spector) 在系统问题上的协助。赵岳感谢菲利普·克雷恩布尔 (Philipp Krühenbühl) 的讨论与反馈。孙宇感谢其博士生导师阿廖沙·埃夫罗斯 (Alyosha Efros) 在机器学习研究中提出的关注像素的深刻建议。

作者署名说明。加松·侯赛因 (Gashon Hussein) 和宋有真 (Youjin Song) 在本项目初版提交至 CVPR 后加入团队, 并对最终版本做出了重大贡献。由于 CVPR 不允许在提交后添加作者, 他们的名字无法出现在 OpenReview 和会议网页上。然而, 我们一致认为官方作者名单应包含他们的名字, 正如我们发布的 PDF 中所呈现的那样。没有他们的工作, 本项目不可能完成。

参考文献

- [1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. *NeurIPS*, 2022. 4
- [2] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024. 9
- [3] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020. 6
- [4] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T Freeman. Maskgit: Masked generative image transformer. In *CVPR*, 2022. 10
- [5] Xinyuan Chen, Yaohui Wang, Lingjun Zhang, Shaobin Zhuang, Xin Ma, Jiashuo Yu, Yali Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. Seine: Short-to-long video diffusion model for generative transition and prediction. In *ICLR*, 2023. 10
- [6] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Banghua Zhu, Hao Zhang, Michael Jordan, Joseph E Gonzalez, et al. Chatbot arena: An open platform for evaluating llms by human preference. In *ICML*, 2024. 2, 8
- [7] Kevin Clark, Kelvin Guu, Ming-Wei Chang, Panupong Paspapat, Geoffrey Hinton, and Mohammad Norouzi. Meta-learning fast weight language models. *EMNLP*, 2022. 9
- [8] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. In *ICML*, 2024. 2, 6, 9
- [9] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *NeurIPS*, 2022. 6
- [10] Songwei Ge, Thomas Hayes, Harry Yang, Xi Yin, Guan Pang, David Jacobs, Jia-Bin Huang, and Devi Parikh. Long video generation with time-agnostic vqgan and time-sensitive transformer. In *ECCV*, 2022. 10
- [11] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Communications of the ACM*, 2020. 10
- [12] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *COLM*, 2024. 2, 9
- [13] Agrim Gupta, Lijun Yu, Kihyuk Sohn, Xiuye Gu, Meera Hahn, Fei-Fei Li, Irfan Essa, Lu Jiang, and José Lezama.

- Photorealistic video generation with diffusion models. In *ECCV*, 2024. 10
- [14] Tanmay Gupta, Dustin Schwenk, Ali Farhadi, Derek Hoiem, and Aniruddha Kembhavi. Imagine this! scripts to compositions to videos. In *ECCV*, 2018. 10
- [15] Yingqing He, Tianyu Yang, Yong Zhang, Ying Shan, and Qifeng Chen. Latent video diffusion models for high-fidelity long video generation. *arXiv preprint arXiv:2211.13221*, 2022. 10
- [16] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016. 3
- [17] Roberto Henschel, Levon Khachatryan, Daniil Hayrapetyan, Hayk Poghosyan, Vahram Tadevosyan, Zhangyang Wang, Shant Navasardyan, and Humphrey Shi. Streamingt2v: Consistent, dynamic, and extendable long video generation from text. *arXiv preprint arXiv:2403.14773*, 2024. 10
- [18] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022. 1
- [19] Wenyi Hong, Ming Ding, Wendi Zheng, Xinghan Liu, and Jie Tang. Cogvideo: Large-scale pretraining for text-to-video generation via transformers. In *ICLR*, 2023. 2, 3
- [20] Ting-Hao Huang, Francis Ferraro, Nasrin Mostafazadeh, Is-han Misra, Aishwarya Agrawal, Jacob Devlin, Ross Girshick, Xiaodong He, Pushmeet Kohli, Dhruv Batra, et al. Visual storytelling. In *NAACL*, 2016. 10
- [21] Kazuki Irie, Imanol Schlag, Róbert Csordás, and Jürgen Schmidhuber. Going beyond linear transformers with recurrent fast weight programmers. *NeurIPS*, 2021. 9
- [22] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of stylegan. In *CVPR*, 2020. 10
- [23] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *ICML*, 2020. 2, 9
- [24] Louis Kirsch and Jürgen Schmidhuber. Meta learning back-propagation and improving it. *NeurIPS*, 34:14122–14134, 2021. 9
- [25] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, Kathrina Wu, Qin Lin, Junkun Yuan, Yanxin Long, Aladdin Wang, Andong Wang, Changlin Li, Duojuan Huang, Fang Yang, Hao Tan, Hongmei Wang, Jacob Song, Jiawang Bai, Jianbing Wu, Jinbao Xue, Joey Wang, Kai Wang, Mengyang Liu, Pengyu Li, Shuai Li, Weiyan Wang, Wenqing Yu, Xincheng Deng, Yang Li, Yi Chen, Yutao Cui, Yuanbo Peng, Zhentao Yu, Zhiyu He, Zhiyong Xu, Zixiang Zhou, Zunnan Xu, Yangyu Tao, Qinglin Lu, Songtao Liu, Dax Zhou, Hongfa Wang, Yong Yang, Di Wang, Yuhong Liu, Jie Jiang, and Caesar Zhong. Hunyuanvideo: A systematic framework for large video generative models. *arXiv preprint arXiv:2412.03603*, 2025. 10
- [26] Yitong Li, Zhe Gan, Yelong Shen, Jingjing Liu, Yu Cheng, Yuexin Wu, Lawrence Carin, David Carlson, and Jianfeng Gao. Storygan: A sequential conditional gan for story visualization. In *CVPR*, 2019. 10
- [27] Shanchuan Lin, Bingchen Liu, Jiashi Li, and Xiao Yang. Common diffusion noise schedules and sample steps are flawed. In *WACV*, 2024. 1
- [28] Chang Liu, Haoning Wu, Yujie Zhong, Xiaoyun Zhang, Yanfeng Wang, and Weidi Xie. Intelligent grimm-open-ended visual storytelling via latent diffusion models. In *CVPR*, 2024. 10
- [29] Adyasha Maharana, Darryl Hannan, and Mohit Bansal. Storydall-e: Adapting pretrained text-to-image transformers for story continuation. In *ECCV*, 2022. 10
- [30] Shentong Mo and Yapeng Tian. Scaling diffusion mamba with bidirectional ssms for efficient image and video generation. *arXiv preprint arXiv:2405.15881*, 2024. 4
- [31] Xichen Pan, Pengda Qin, Yuhong Li, Hui Xue, and Wenhui Chen. Synthesizing coherent story with auto-regressive latent diffusion models. In *WACV*, 2024. 10
- [32] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *CVPR*, 2023. 3, 4
- [33] Tanzila Rahman, Hsin-Ying Lee, Jian Ren, Sergey Tulyakov, Shweta Mahajan, and Leonid Sigal. Make-a-story: Visual memory conditioned consistent story generation. In *CVPR*, 2023. 10
- [34] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *ICLR*, 2022. 1
- [35] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *ICML*, 2021. 2, 9
- [36] Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992. 9
- [37] Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992. 2, 9
- [38] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024. 1
- [39] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019. 6
- [40] Ivan Skorokhodov, Sergey Tulyakov, and Mohamed Elhoseiny. Stylegan-v: A continuous video generator with the price, image quality and perks of stylegan2. In *CVPR*, 2022. 10
- [41] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *ICLR*, 2021. 1
- [42] Benjamin F Spector, Simran Arora, Aaryan Singhal, Daniel Y Fu, and Christopher Ré. Thunderkittens: Simple, fast, and adorable ai kernels. In *ICLR*, 2025. 1
- [43] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): Rnns with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024. 2, 3, 5, 6, 9, 1
- [44] The Movie Gen team. Movie gen: A cast of media foundation models. *arXiv preprint arXiv:2410.13720*, 2024. 2, 6, 8, 10

- [45] Ruben Villegas, Mohammad Babaeizadeh, Pieter-Jan Kindermans, Hernan Moraldo, Han Zhang, Mohammad Taghi Saffar, Santiago Castro, Julius Kunze, and Dumitru Erhan. Phenaki: Variable length video generation from open domain textual description. In *ICLR*, 2023. [10](#)
- [46] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *ICML*, 2008. [3](#)
- [47] Hongjie Wang, Chih-Yao Ma, Yen-Cheng Liu, Ji Hou, Tao Xu, Jialiang Wang, Felix Juefei-Xu, Yaqiao Luo, Peizhao Zhang, Tingbo Hou, Peter Vajda, Niraj K. Jha, and Xiaoliang Dai. Lingen: Towards high-resolution minute-length text-to-video generation with linear computational complexity, 2024. [2](#)
- [48] Ke Alexander Wang, Jiaxin Shi, and Emily B Fox. Test-time regression: a unifying framework for designing sequence models with associative memory. *arXiv preprint arXiv:2501.12352*, 2025. [9](#)
- [49] Xintao Wang, Liangbin Xie, Chao Dong, and Ying Shan. Real-esrgan: Training real-world blind super-resolution with pure synthetic data. In *ICCVW*, 2021. [5](#)
- [50] Yaohui Wang, Xinyuan Chen, Xin Ma, Shangchen Zhou, Ziqi Huang, Yi Wang, Ceyuan Yang, Yinan He, Jiashuo Yu, Peiqing Yang, et al. Lavie: High-quality video generation with cascaded latent diffusion models. *IJCV*, 2024. [10](#)
- [51] Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajwal Bhargava, Rui Hou, Louis Martin, Rashmi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, et al. Effective long-context scaling of foundation models. In *NAACL*, 2024. [5](#)
- [52] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *NeurIPS*, 2024. [6](#), [9](#)
- [53] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *ICLR*, 2025. [2](#), [6](#)
- [54] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, et al. Cogvideox: Text-to-video diffusion models with an expert transformer. In *ICLR*, 2025. [2](#), [10](#), [1](#)
- [55] Shengming Yin, Chenfei Wu, Huan Yang, Jianfeng Wang, Xiaodong Wang, Minheng Ni, Zhengyuan Yang, Linjie Li, Shuguang Liu, Fan Yang, et al. Nuwa-xl: Diffusion over diffusion for extremely long video generation. *arXiv preprint arXiv:2303.12346*, 2023. [10](#)
- [56] Yupeng Zhou, Daquan Zhou, Ming-Ming Cheng, Jiashi Feng, and Qibin Hou. Storydiffusion: Consistent self-attention for long-range image and video generation. In *NeurIPS*, 2024. [10](#)

Video len.	Ctx. len	Trainable parameters	Learning rate	Schedule	Steps
3 sec	18048	TTT / Pre-trained Params	$1 \times 10^{-4} / 1 \times 10^{-5}$	Cosine / Constant	5000
9 sec	51456	TTT + Local Attn (QKVO)	1×10^{-5}	Constant	5000
18 sec	99894	TTT + Local Attn (QKVO)	1×10^{-5}	Constant	1000
30 sec	168320	TTT + Local Attn (QKVO)	1×10^{-5}	Constant	500
63 sec	341550	TTT + Local Attn (QKVO)	1×10^{-5}	Constant	250

表 2. 多阶段微调的超参数。首先，整个预训练模型在《猫和老鼠》的 3 秒片段上进行微调，为新引入的测试时训练（TTT）层和门控分配更高的学习率。然后，仅以降低的学习率微调测试时训练层、门控和自注意力参数。

	Text following	Motion naturalness	Aesthetics	Temporal consistency	Average
Local Attention	965	972	969	944	962
TTT-Linear	1003	995	1007	1001	1001
Mamba 2	1023	987	1008	1004	1005
Gated DeltaNet	1020	1039	1044	1026	1032
SWA	995	1004	993	980	993
TTT-MLP	994	1002	1002	1019	1004

表 3. 18 秒视频的人工评估结果，详见第 4.3 节和第 4.4 节。

A. 实验细节

扩散时间表。遵循 CogVideoX [54]，本文采用 v 预测（ v -prediction）[34] 对模型进行微调，该过程包含 1000 步的扩散噪声调度，并在最后一步强制实施零信噪比（Zero-SNR）[27]。

训练配置。所有训练阶段均采用以下超参数：

- **优化器：**AdamW 配合 $(\beta_1, \beta_2) = (0.9, 0.95)$
- **学习率：**前 2% 训练步数内线性预热
- **批大小：**64
- **梯度裁剪：** $10^{-4} \times 0.1$
- **权重衰减：**应用于除偏差外的所有参数及归一化层
- **变分自编码器缩放因子：**1.0
- **随机失活：**以概率 0.1 将文本提示置零
- **精确率：**使用 PyTorch FSDP2 的混合精度

TTT 配置。TTT 层的一个关键超参数是内环学习率 η ，对于 TTT-Linear 我们设为 $\eta = 1.0$ ，对于 TTT-MLP 设为 $\eta = 0.1$ 。

采样调度。我们遵循 DDIM 采样器 [41]，共 50 步，应用动态无分类器引导

（CFG）[18]，将 CFG 强度从 1 增加到 4，并利用负提示进一步提升视频质量。

B. 片上张量并行细节

本文采用雷核（ThunderKittens）[42] 实现第 3.5 小节所述的 TTT-MLP 核函数。

隐藏状态分片。本文遵循张量并行的标准策略，对第一层按列分片，对第二层按行分片。由于高斯误差线性单元（GeLU）非线性是逐元素的，TTT 层的前向传播仅需一次归约即可计算用于更新隐藏状态的内部损失。

进一步延迟优化。本文借鉴闪电注意力 3（FlashAttention-3）[38] 中的多项技术，以进一步降低英伟达 Hopper 架构 GPU 上的输入输出延迟。具体而言，本文实现了一种多级流水线方案，该方案从高带宽存储器（HBM）异步预取未来的小批量数据，使数据传输与当前小批量的计算重叠。这种方法被称为生产者-消费者异步，涉及将专用线程组分配给数据加载（生产者）或计算（消费者）。

梯度检查点。本文将沿序列维度的梯度检查点 [43] 直接集成到融合核函数中。为减少输入输出引起的停顿和统一计算设备架构（CUDA）线程的工作负载，本文使用张量内存加速器

（张量内存加速器）用于执行异步内存存储。

Format 1

Tom is happily eating an apple pie at the kitchen table. Jerry looks longingly wishing he had some. Jerry goes outside the front door of the house and rings the doorbell. While Tom comes to open the door, Jerry runs around the back to the kitchen. Jerry steals Tom's apple pie. Jerry runs to his mouse hole carrying the pie, while Tom is chasing him. Just as Tom is about to catch Jerry, Jerry makes it through the mouse hole and Tom slams into the wall.

Format 2

Segment 1-2: Tom walks into the kitchen carrying an apple pie. He sits at the table and begins eating.

Segment 3-5: The viewpoint shifts behind the countertop, revealing Jerry hiding behind a salt shaker. Jerry steps out, watches Tom eating the pie, and eagerly rubs his tummy. He then darts off-screen to the right.

Segment 6-8: Outside the house, Jerry approaches the front door, jumps to press the doorbell, and quickly runs away.

The story continues...

Format 3

<start_scene>The kitchen has soft yellow walls, white cabinets, and a window with red-and-white checkered curtains letting in gentle sunlight. In the middle, there's a round wooden table with matching chairs, sitting on a clean white-tiled floor. Tom, the blue-gray cat, walks in from the left holding a warm, golden-brown pie on a shiny silver tray. He moves calmly across the room toward the table, carefully places the pie down, pulls out a chair, and sits comfortably. The camera smoothly follows Tom from left to right, clearly showing each of his movements.

The kitchen has soft yellow walls, white cabinets, and a window with red-and-white checkered curtains letting in gentle sunlight. In the middle, there's a round wooden table with matching chairs, sitting on a clean white-tiled floor. Tom, the blue-gray cat, sits comfortably at the table with the golden-brown pie resting on its shiny silver tray directly in front of him. He carefully uses his paw to pick up a slice from the tray, lifts it toward his mouth, and takes a large bite. The camera slowly moves closer, clearly showing Tom enjoying his pie as crumbs lightly fall onto the table.<end_scene>

<start_scene>The kitchen has soft yellow walls, white cabinets, and a window with red-and-white checkered curtains letting in gentle sunlight. The cabinets have white countertops, on which a tall glass salt shaker is sitting. In the background, Tom, the blue-gray cat, sits at the round wooden table, eating the golden-brown pie. Jerry, the brown mouse, stands on the white countertop, hidden behind the salt shaker. Jerry glances around briefly, then steps out from behind the salt shaker. The camera captures Jerry as he emerges from behind the salt shaker and stands on the countertop.

The kitchen has soft yellow walls, white cabinets, and a window with red-and-white checkered curtains letting in gentle sunlight. The cabinets have white countertops, on which a tall glass salt shaker is sitting. In the background, Tom, the blue-gray cat, sits at the round wooden table, eating the golden-brown pie. Jerry, the brown mouse, stands on top of the countertop next to the salt shaker. Jerry rubs his stomach with his paws and looks at Tom, the blue-gray cat. The camera remains in position slightly to the side of Jerry, capturing his hungry expression.

The kitchen has soft yellow walls, white cabinets, and a window with red-and-white checkered curtains letting in gentle sunlight. The cabinets have white countertops, on which a tall glass salt shaker is sitting. In the background, Tom, the blue-gray cat, sits at the round wooden table, eating the golden-brown pie. Jerry, the brown mouse, stands on top of the countertop next to the salt shaker. Jerry gives his belly a final rub, then turns to the left and quickly begins running along the countertop toward the right side of the scene. The camera captures Jerry as he disappears off-screen.<end_scene>

<start_scene>The front of the house has light blue walls, a white wooden front door, a small round white doorbell button beside it, and a small porch with steps leading down to a neat green lawn. Bright flowers in red and yellow line the walkway, and sunlight warmly fills the area. Jerry, the brown mouse, calmly walks up onto the porch from the right side, moving toward the front door and doorbell at the center of the scene. The camera smoothly tracks Jerry's steps, capturing clearly as he crosses the porch and comes to a gentle stop near the steps, glancing cautiously upward at the doorbell.

The story continues...

图 8. 第 3.2 小节讨论的三种提示格式示意图：（1）情节简短摘要，（2）片段级句子描述，（3）详细分镜脚本。